

Processor Allocation

Drone Remote Weapon Station — Aleppo Systems / RDS
MIL-STD-882E | Document ref: Softwaregeneralarchitecture

1. Summary

Unit	Processor	OS / Runtime	Integrity Level
FCC	NVIDIA Jetson Orin NX	Ubuntu Linux (RT kernel)	Medium
ISM	STM32H7 or TI TMS570 (dedicated MCU)	Bare-metal / FreeRTOS / SafeRTOS	High
GUI / HMI	GCS laptop / tablet	Windows / Linux / Browser	Low

2. FCC — NVIDIA Jetson Orin NX

Why Jetson fits the FCC role

The FCC is **Medium integrity** software responsible for non-safety-critical but computationally demanding tasks. The Jetson Orin NX is purpose-built for exactly this profile:

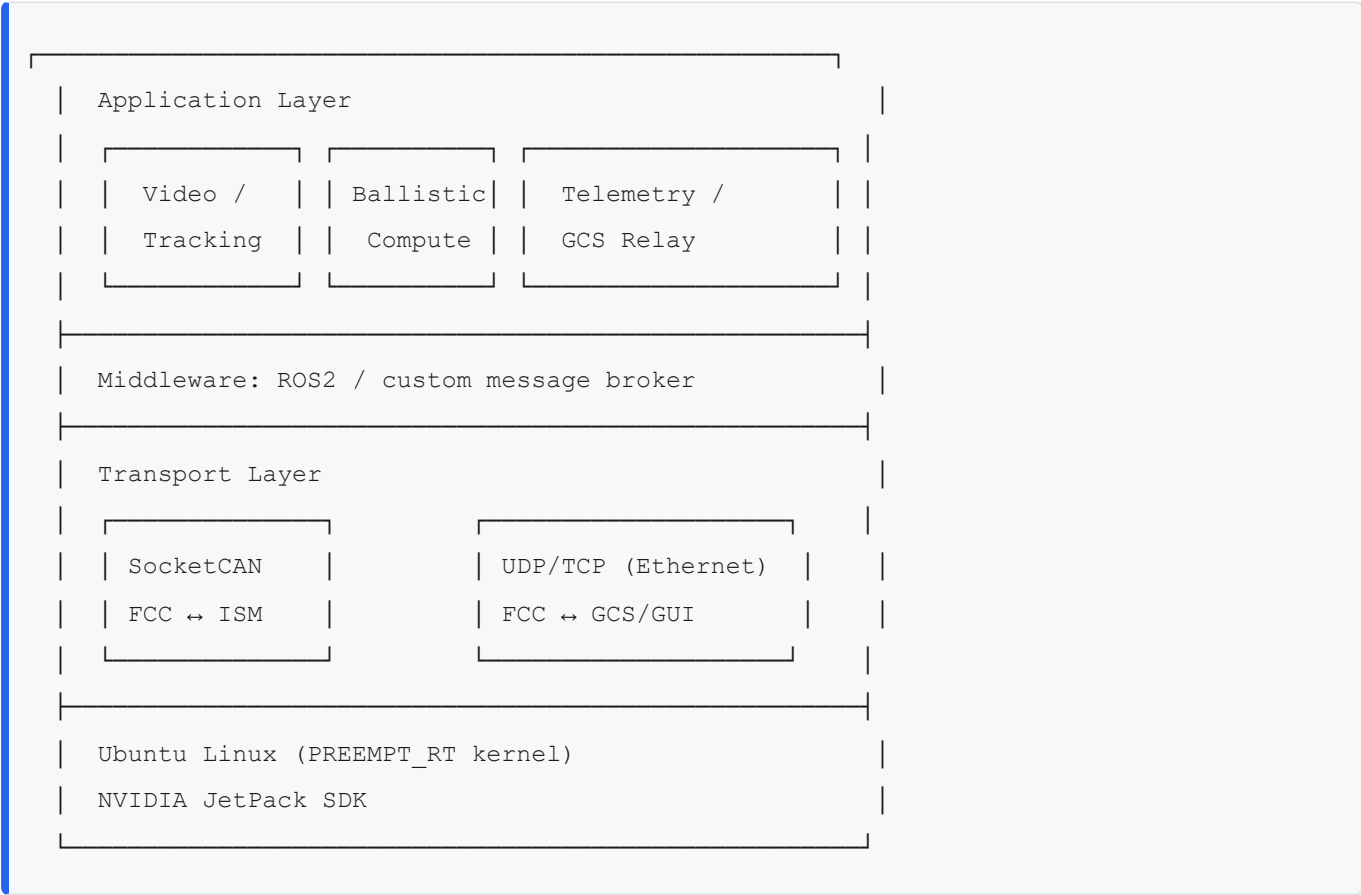
Capability	FCC Requirement	Jetson Orin NX
Video processing & streaming	Real-time encode/decode	1x 8K / 4x 4K NVENC, NVDEC
Aiming & stabilisation algorithms	High-throughput compute	1024-core Ampere GPU
Ballistic computation	Floating point intensive	ARM Cortex-A78AE CPU cluster
Tracking assist	Neural inference	32 TOPS DLA + GPU
GCS RF link	Gigabit Ethernet	1x GbE, PCIe for RF module
ISM interface	CAN bus controller	2x CAN FD (on carrier board)
Linux environment	ROS2 / middleware support	Ubuntu 20.04 / 22.04

Jetson Orin NX Key Specs

- CPU:** 8-core ARM Cortex-A78AE v8.2 64-bit
- GPU:** 1024-core NVIDIA Ampere
- Memory:** 8 GB / 16 GB LPDDR5

- **Storage:** NVMe SSD via M.2
- **Interfaces:** GbE, USB 3.2, PCIe Gen4, I2C, SPI, UART, CAN FD (via carrier)
- **Power:** 10–25 W (configurable)
- **OS:** Ubuntu Linux with optional real-time kernel patch (PREEMPT_RT)

FCC Software Stack on Jetson



3. ISM — Dedicated Microcontroller (STM32H7 / TI TMS570)

Why Jetson cannot run the ISM

The ISM is **High integrity (MIL-STD-882E)**. Running it on a Jetson/Linux would violate core safety principles:

Issue	Impact
Linux is non-deterministic	Kernel scheduler can delay ISM watchdog tick by milliseconds — unacceptable for safety timing
Complex OS attack surface	Kernel panic, driver fault, or OOM killer could silence the ISM entirely
Shared hardware resources	GPU/CPU contention from FCC video tasks can starve ISM threads
Not certifiable	Linux cannot be certified to IEC 61508 SIL-2/3 or MIL-STD-882E High integrity without enormous qualification effort
SWR-SAFE-005 violation	ISM and non-safety software (video, logging, GUI) must be physically separated — not just OS-level isolated

SWR-SAFE-005: *"Safety-critical software shall be separated from non-safety services so that video, logging, or GUI failure cannot block torque disable."*
Running ISM on the same Jetson as video/logging directly violates this requirement.

Recommended ISM Processors

Option A — STM32H7 (recommended for cost/ecosystem)

Property	Value
Core	ARM Cortex-M7, 480 MHz
Flash	Up to 2 MB
RAM	1 MB (TCM for deterministic access)
CAN	2x FDCAN (ISO 11898)
UART	Up to 8x
Watchdog	IWDG (independent) + WWDG
OS	Bare-metal or FreeRTOS / SafeRTOS
Certification path	IEC 61508 with SafeRTOS; well-documented safety MCU
Power	~300 mW active
Cost	Low

Option B — TI TMS570LS (highest safety pedigree)

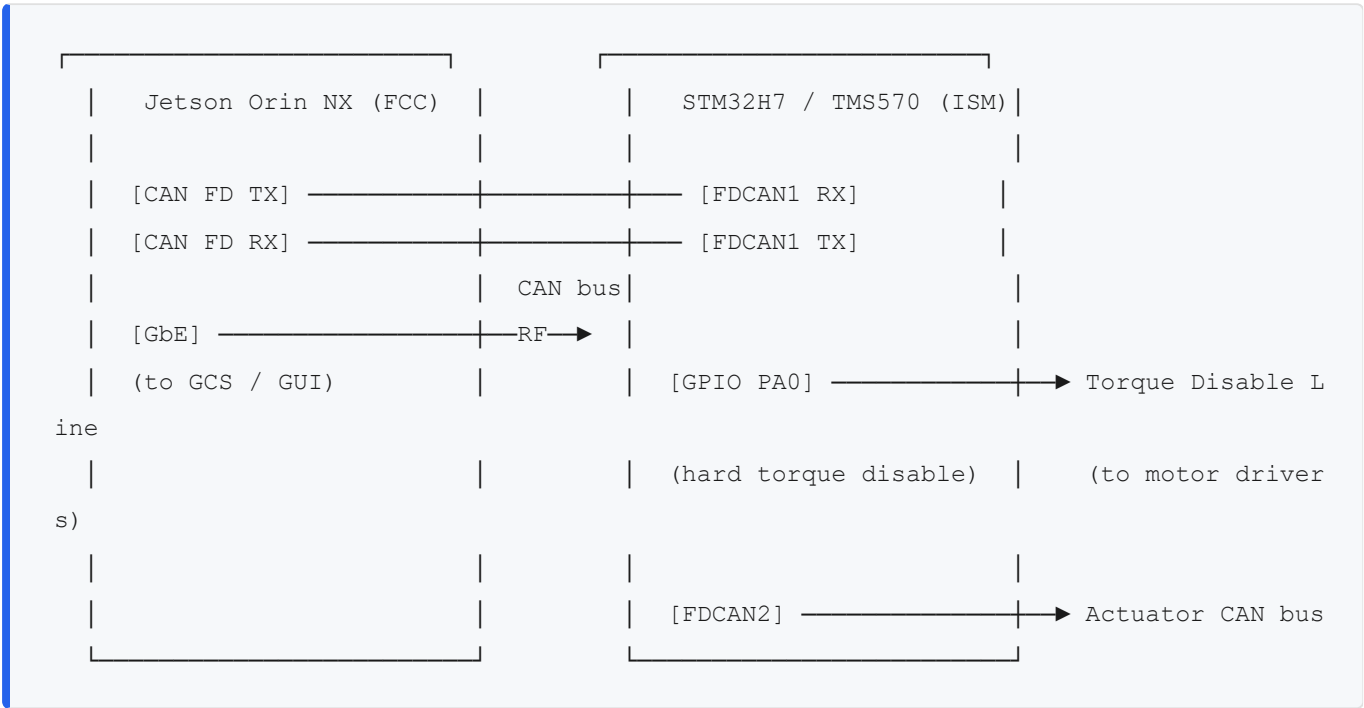
Property	Value
Core	Dual-core ARM Cortex-R5F (lockstep)
Flash	Up to 4 MB
CAN	3x DCAN
Watchdog	Dual independent watchdogs
OS	Bare-metal / TI SafeRTOS / AUTOSAR
Certification path	IEC 61508 SIL-3, ISO 26262 ASIL-D ready
Special feature	CPU lockstep — both cores run in parallel, hardware compares outputs cycle-by-cycle; any discrepancy → safe state
Power	~500 mW active
Cost	Higher

Recommendation: STM32H7 for a first prototype. Migrate to TMS570 if formal safety certification (IEC 61508 SIL-2+) is required.

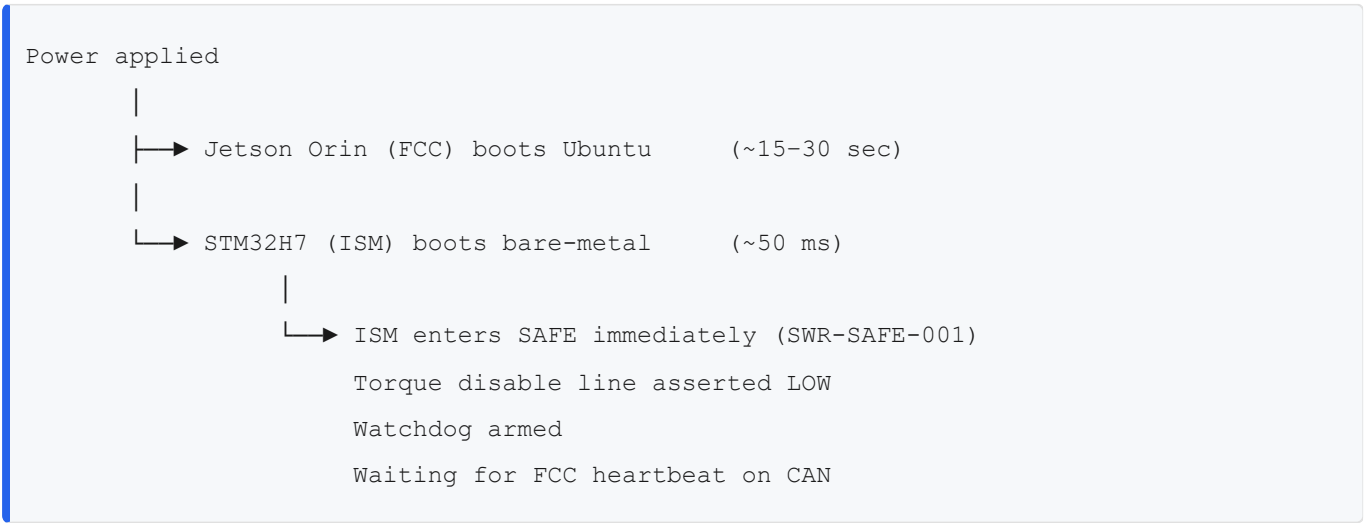
ISM Software Stack on STM32H7



4. Physical Wiring — On-Platform



5. Boot & Power-Up Sequence



*The ISM is operational and enforcing SAFE **before the FCC finishes booting**.
The system cannot become unsafe during the FCC boot window.*

6. Development & Debug

Task	Tool
FCC development	SSH into Jetson, build with CMake/GCC, debug with GDB over network
ISM development	STM32CubeIDE or VS Code + OpenOCD, SWD debug via ST-Link
CAN bus monitoring	<code>candump can0</code> on Jetson; CANalyzer / PCAN-View on bench
ISM unit tests (this repo)	Cross-compiled for x86 Linux / WSL — run without hardware

7. Why Jetson Cannot Run the ISM — Detailed Justification

This section documents the architectural decision to **reject the NVIDIA Jetson as an ISM processor**, even though it is used for the FCC. This question is expected to arise during design reviews and safety audits.

7.1 Linux Is Non-Deterministic

The Jetson runs Ubuntu Linux. The Linux kernel scheduler can preempt any thread at any moment and delay its execution by several milliseconds — or longer under load. The ISM requires a guaranteed 1 ms tick to kick the hardware watchdog and re-assert the torque-disable line.

Linux scheduler worst-case preemption delay:

1 ms - 10 ms (without PREEMPT_RT)

~100 μs - 1 ms (with PREEMPT_RT)

ISM required watchdog kick interval:

1 ms

IWDG hardware timeout:

26 ms

→ Under Linux, a single heavy GPU/video burst on the FCC could delay the ISM watchdog kick past the IWDG timeout → unintended MCU reset.

→ On bare-metal STM32H7, the TIM6 IRQ fires at exactly 1 ms ± nanoseconds.

7.2 Boot Time — System Is Unsafe During Jetson Boot

Processor	Time to SAFE state	Torque-disable asserted
STM32H7 (bare-metal)	~15 ms	T+5 ms from power-on
Jetson Orin NX (Linux)	~30 seconds	Only after OS + app boots

If the ISM ran on the Jetson, the weapon station actuators would be **ungated for up to 30 seconds** after every power cycle — a direct violation of SWR-SAFE-001 (SAFE on power-up).

7.3 Physical Independence Requirement (SWR-SAFE-005)

**"Safety-critical software shall be separated from non-safety services so that video, logging, or GUI failure cannot block torque disable."* — SWR-SAFE-005*

If ISM and FCC share the same Jetson:

Failure scenario	Effect on Jetson-hosted ISM
GPU driver crash (OOM killer)	ISM process killed → torque not disabled
Video encode overload	CPU saturated → ISM tick delayed → watchdog fires unexpectedly
FCC software bug causing infinite loop	ISM starved of CPU time
Kernel panic	Both FCC and ISM down simultaneously — no safety authority

On a dedicated STM32H7, **none of these can happen** — the ISM MCU has no GPU, no video, no Linux, no shared resources with FCC.

7.4 Certifiability Under MIL-STD-882E High Integrity

MIL-STD-882E High integrity software requires demonstrating that every execution path is known, bounded, and cannot be interrupted by unintended software behavior. The Linux kernel contains approximately **30 million lines of code**. Qualifying it to High integrity is not practically achievable.

Software	Lines of code	Qualifiable to High integrity?
Ubuntu Linux kernel	~30,000,000	✗ No
FreeRTOS kernel	~10,000	⚠ Possible with effort
SafeRTOS kernel	~10,000	✓ Pre-certified IEC 61508 SIL-3
ISM bare-metal (this codebase)	~2,000	✓ Fully auditable

7.5 Power Consumption

The ISM must remain operational even if the platform power budget is under stress (e.g., damaged power supply, battery depletion).

Processor	Active power
Jetson Orin NX	10 – 25 W
STM32H7	~300 mW

The STM32H7 can remain operational on a small backup capacitor or battery if main power fails — giving the ISM time to assert SAFE before power is fully lost.

7.6 Summary — Processor Allocation Decision

DECISION: Jetson Orin NX is approved for FCC ONLY.

The ISM SHALL run on a dedicated bare-metal MCU (STM32H7 or TMS570). This is a safety-critical architectural requirement derived from SWR-SAFE-001, SWR-SAFE-002, and SWR-SAFE-005.

This decision SHALL NOT be changed without a formal safety impact assessment and updated hazard analysis.

Criterion	Jetson (Linux)	STM32H7 (Bare-metal)
Deterministic 1ms tick	✗ No	✓ Absolute
Time to SAFE from power-on	✗ ~30 seconds	✓ ~15 ms
Physical separation from FCC	✗ Same processor	✓ Separate hardware
Immune to FCC software crash	✗ No	✓ Yes
Auditable codebase	✗ 30M LOC kernel	✓ ~2K LOC ISM
MIL-STD-882E High integrity	✗ Not achievable	✓ Yes
Power consumption	✗ 10–25 W	✓ ~300 mW
Cost	✗ High	✓ Low

8. References

- NVIDIA Jetson Orin NX — *Product Brief and Technical Reference Manual*
- STMicroelectronics — *STM32H7 Reference Manual (RM0433)*
- Texas Instruments — *TMS570LS Safety MCU Datasheet*
- MIL-STD-882E — *Standard Practice for System Safety*
- IEC 61508 — *Functional Safety of E/E/PE Safety-related Systems*